Лекция 1. Введение в параллелизм

На этой лекции вводятся основные понятия, используемые в параллельном программировании. Начнем с попытки осмыслить само понятие компьютерной обработки данных. Затем обратимся к архитектурам компьютерных систем. В завершение параграфа обсудим принципы, которыми будем руководствоваться при анализе параллельных алгоритмов.

Категории компьютерных систем

Компьютерные системы можно разделить на четыре основных категории. Для этого нужно несколько сменить наше представление о том, как работает программа. С точки зрения центрального процессора программа представляет собой поток инструкций, которые следует расшифровать и выполнить. Данные тоже можно считать поступающими в виде потока. Четыре категории, о которых мы говорим, определяются тем, поступают ли программы и данные одним потоком или несколькими.

Один поток инструкций / один поток данных

Модель «один поток инструкций / один поток данных» (*SISD*) представляет собой классическую модель с одним процессором. К ней относятся как компьютеры предыдущих поколений, так и многие современные компьютеры. Процессор такого компьютера способен выполнять в любой момент времени лишь одну инструкцию и работать лишь с одним набором данных. В таких последовательных системах никакого параллелизма нет в отличие от прочих категорий.

Один поток инструкций / несколько потоков данных

В компьютерах с одним потоком инструкций и несколькими потоками данных (*SIMD*) имеется несколько процессоров, выполняющих одну и ту же операцию, но над различными данными. *SIMD*-машины иногда называются также векторными процессорами, поскольку они хорошо приспособлены для операций над векторами, в которых каждому процессору передается одна координата вектора и после выполнения операции весь вектор оказывается обработанным. Например, сложение

векторов – покоординатная операция. Первая координата суммы векторов – сумма первых координат суммируемых векторов, вторая – сумма вторых координат, и т. д. В нашей *SIMD*-машине каждый процессор получит инструкцию сложить пару координат входных векторов. После выполнения этой единственной инструкции результат будет подсчитан полностью. Заметим, что на векторе из N элементов *SISD*-машине потребуется выполнить N итераций цикла, а *SIMD*-машине с не менее чем N процессорами, хватит одной операции.

Несколько потоков инструкций / один поток данных

Возможность одновременно выполнять различные операции над одними и теми же данными может поначалу показаться странной: не так уж часто встречаются программы, в которых нужно какое-то значение возвести в квадрат, умножить на 2, вычесть из него 10 и т. д. Однако, если посмотреть на эту ситуацию с другой точки зрения, то можно увидеть, что на машинах такого типа можно усовершенствовать проверку числа на простоту. Если число процессоров равно N, то можно проверить простоту любого числа между 1 и N^2 на *MISD*-машине за одну операцию: если число X составное, то у него должен быть делитель, не превосходящий $\sqrt{X}$. Для проверки простоты числа $X \le N^2$ поручим первому процессору делить на 2, второму – делить на 3, третьему – на 4, и так до процессора с номером $K - 1$, который будет делить на K, где $K = \left\lfloor \sqrt{X} \right\rfloor$. Если на одном из этих процессоров деление нацело проходит успешно, то число X составное. Поэтому результат достигается за одну операцию. Как нетрудно видеть, на последовательной машине выполнение этого алгоритма потребовало бы по меньшей мере $\left\lfloor \sqrt{X} \right\rfloor$ проходов, на каждом из которых производится одно деление.

Несколько потоков инструкций / несколько потоков данных

Это наиболее гибкая из категорий. В случае *MIMD*-систем мы имеем дело с несколькими процессорами, каждый из которых способен выполнять свою инструкцию. Кроме того имеется несколько потоков данных, и каждый процессор может работать со своим набором данных. На практике это означает, что *MIMD*-система может выполнять на каждом процессоре свою программу или отдельные части одной и той же программы, или векторные операции так же, как и *SIMD*-

конфигурация. Большинство современных подходов к параллелизму, включая кластеры компьютеров и мультипроцессорные системы, относятся к категории *MIMD*.

Параллельные архитектуры

В архитектуре параллельных компьютерных систем основную роль играют два аспекта: как связаны между собой процессоры и их память и как процессоры взаимодействуют. Об этих аспектах мы и будем говорить при обсуждении параллельных алгоритмов, поскольку те или иные решения могут иметь разную эффективность для различных задач.

Слабо и сильно связанные машины

В машинах со слабой связью у каждого процессора есть своя собственная память, а связь между процессорами осуществляется по «сетевым» кабелям. Это архитектура компьютерных кластеров: каждый компьютер кластера представляет собой отдельную компьютерную систему и может работать самостоятельно. Параллелизм осуществляется за счет распределения заданий по компьютерам кластера центральным управляющим компьютером.

В машинах с тесными связями все процессоры пользуются общей центральной памятью. Взаимодействие процессоров осуществляется за счет того, что один из них записывает информацию в общую память, а остальные считывают ее оттуда.

Взаимодействие процессоров

Мы говорили о том, что в машинах со слабыми связями взаимодействие процессоров осуществляется по кабелям или проводам. Посмотрим на возможную организацию таких связей. На одном конце спектра расположена полносвязная сеть, каждый процессор в которой соединен со всеми остальными. На другом конце — линейная сеть, в которой процессоры выстроены в цепочку и каждый процессор, за исключением двух концевых, соединен с двумя соседними (у концевых процессоров по одному соседу). Информация в полносвязной сети передается от процессора к процессору очень быстро, однако такая организация требует большой протяженности кабеля. В линейной сети информация передается

медленнее за счет необходимости пропускать ее через промежуточные узлы, и разрыв цепочки в одном месте обрывает поток данных.

В качестве альтернативы линейной сети, повышающей устойчивость, можно предложить кольцевую структуру; она устроена так же, как и линейная сеть, но первое и последнее звено цепочки тоже соединены между собой. В такой сети информация распространяется быстрее, поскольку она должна проходить уже не более половины процессоров.

Скажем, в линейной сети на рис. 1 сообщение из узла 1 в узел 5 должно пройти через три промежуточных узла, а в кольцевой сети на рис. 2 достаточно пропустить его через единственный промежуточный узел 6. В сети на рис. 3 процессоры расположены в вершинах двумерной решетки, а кабели соединяют соседей по горизонтали или по вертикали. Такая сеть еще более эффективна и устойчива, чем кольцевая, однако расход кабеля в ней возрастает.

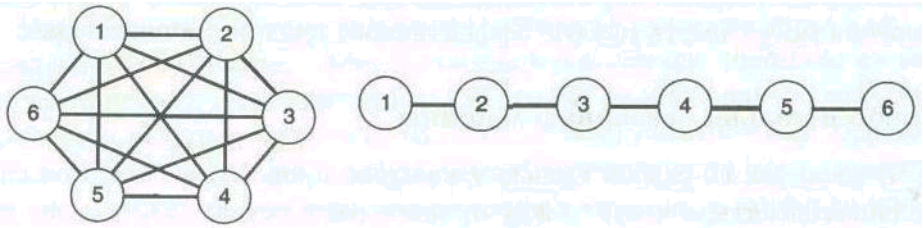


Рис. 1. Полносвязная и линейная конфигурации сети

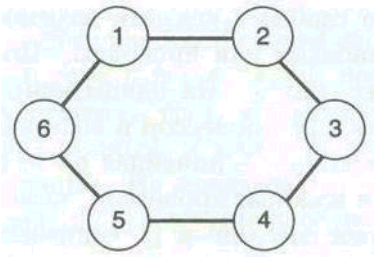


Рис. 2. Кольцевая конфигурация сети

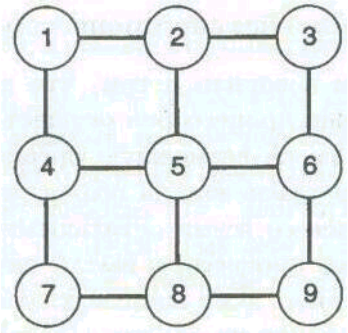


Рис. 3. Решетчатая сеть

Есть и другие возможности, к ним относятся древовидные сети, процессоры в которых образуют бинарные деревья, а также гиперкубы, обобщающие двумерную решетку на три и более измерений.

Принципы анализа параллельных алгоритмов

При работе с параллельными алгоритмами нас будут интересовать два новых понятия: коэффициент ускорения и стоимость. Коэффициент ускорения параллельного алгоритма показывает, насколько он работает быстрее оптимального последовательного алгоритма. Так оптимальный алгоритм сортировки требует $O(N \log N)$ операций. У параллельного алгоритма сортировки со сложностью $O(N)$ коэффициент ускорения составил бы $O(\log N)$.

Второе интересующее нас понятие – стоимость параллельного алгоритма, которая определяется как произведение сложности алгоритма на число используемых процессоров. Если в нашей ситуации алгоритм параллельной сортировки за $O(N)$ операций требует столько же процессоров, каково число входных записей, то его стоимость равна $O(N^2)$. Это означает, что алгоритм параллельной сортировки обходится дороже, поскольку стоимость алгоритма последовательной сортировки на одном процессоре совпадает с его сложностью и равна поэтому $O(N \log N)$.

Близким понятием является расщепляемость задачи. Если единственная возможность для нашего алгоритма параллельной сортировки состоит в том, что число процессоров должно равняться числу входных записей, то такой алгоритм становится бесполезным как только число входных записей оказывается достаточно большим. Никаких похожих ограничений в алгоритме последовательной сортировки нет. По большей части нас будут интересовать такие алгоритмы параллельной сортировки, в которых число процессоров значительно меньше потенциального объема входных данных и это число не требует увеличения при росте длины входа.

Модель *PRAM*

При разработке параллельных алгоритмов мы опираемся на одну из четырех рассматриваемых нами моделей компьютерных систем. Одна из проблем

параллельных систем состоит в чтении данных из памяти и записи в память. Что произойдет, например, если два процессора попытаются записать данные в одно и то же место общей памяти?

В рассмотренных ранее последовательных алгоритмах предполагалось, что машина, на которой они реализуются, обладает прямым доступом к любой наперед заданной ячейке памяти (*RAM*). Алгоритмы этой главы будут основаны на параллельном варианте такой машины, называемом *PRAM*. Процессоры нашей *PRAM*-машины тесно связаны между собой и пользуются общим блоком памяти. В каждом процессоре есть несколько регистров, в которых может храниться небольшой объем данных, однако основная часть данных содержится в общей памяти.

Помимо требования тесной связи между процессорами также будем предполагать, что все они осуществляют один и тот же цикл обработки, состоящий в чтении данных из памяти, выполнении операции над ними и записи результата в память. Это означает, что все процессоры одновременно читают память, одновременно обрабатывают прочитанные данные и одновременно производят запись. Спор за ячейки памяти может возникнуть как при чтении, так и при записи. Требование трехшагового цикла означает, что можно не беспокоиться, что произойдет, если процессор *Y* меняет содержимое ячейки памяти в то время как процессор *X* обрабатывает только что прочитанные оттуда данные. Не может также возникнуть и ситуация, в которой один процессор читает содержимое ячейки памяти в тот момент, когда другой пытается что-либо записать в нее.

Споры можно разрешать, предоставляя либо конкурентное, либо исключительное право доступа к памяти. При конкурентном доступе к одной и той же ячейке памяти может обращаться одновременно несколько процессоров. При исключительном доступе к данной ячейке памяти в каждый конкретный момент может обращаться только один процессор, а попытка двух одновременных обращений приводит к появлению сообщения об ошибке. Конкурентный доступ во время чтения не составляет проблемы. Нам, однако, понадобятся и алгоритмы, работающие с исключительным правом чтения. Если при исключительном праве чтения одновременно к одной и той же ячейке памяти обращается несколько процессоров, то возникает ошибка.

При записи также имеется проблема выбора между исключительным и конкурентным доступом. При исключительном доступе право записи в каждую ячейку памяти предоставляется одному процессору, а при попытке записи со стороны нескольких процессоров возникает ошибка. Однако два процессора могут вести запись в разные ячейки памяти одновременно. При конкурентном доступе задача оказывается более сложной: необходимо уметь разрешать возникающие конфликты. В модели с приоритетами каждому процессору приписывается приоритет, и право на запись предоставляется конкурирующему процессору с наивысшим приоритетом. В простейшем варианте этой модели приоритет процессора совпадает с его номером: чем меньше номер, тем выше приоритет. Если, например, в одну ячейку памяти пытаются записывать процессоры номер 4 и номер 7, то предпочтение будет отдано процессору 4. В модели с произвольным доступом будет выбран любой из конкурирующих процессоров – заранее неизвестно какой. В обычной модели допускается одновременная запись, однако только в случае, когда все записываемые значения совпадают. В комбинированной модели система выполняет некоторые операции над записываемыми значениями: может быть записана их сумма, их произведение, наибольший или наименьший элемент, либо результат логической операции (и, или, исключительное или). В различных обстоятельствах любая из этих возможностей может оказаться полезной.

Тем самым у нас есть четыре комбинации возможностей чтения и записи: конкурентное чтение / конкурентная запись (*CRCW*), конкурентное чтение / исключительная запись (*CREW*), исключительное чтение / конкурентная запись (*ERCW*), исключительное чтение / исключительная запись (*EREW*).

Простые параллельные операции

Рассмотрим теперь две элементарные операции: распределение входных данных по процессорам и поиск минимального или максимального элемента списка. Ниже мы обозначаем *i*-ый процессор через P_i, число процессоров через p, число элементов во входном списке через N, а j-ую ячейку памяти через M_j. Выполняемые параллельно операции записываются в скобках *Parallel Start* и *Parallel End*. Если сначала параллельно выполняется один блок операций, а затем другой, то они заключаются в отдельные пары скобок.

Распределение данных в модели *CREW PRAM*

Напомним, что в модели *CREW* одну и ту же ячейку памяти могут читать одновременно несколько процессоров. В результате значение можно занести во все процессоры очень быстро:

P[1] записывает значение в *M*[1]
Parallel Start
    for k = 2 *to p do*
        P[*k*] читает значение из *M*[1]
    end for
Parallel End

Распределение совершается в два этапа. На первом значение записывается в память, на втором – все процессоры читают его. Такая скорость возможна только благодаря конкурентному чтению. Посмотрим теперь, как должна выглядеть эта процедура для модели с исключительным чтением.

Распределение данных в модели *EREW PRAM*

В модели с исключительным чтением значение, записанное процессором P_i, может прочитать лишь один процессор. Если просто организовать цикл поочередного чтения остальными процессорами, то получим последовательный алгоритм, нивелирующий все преимущества параллелизма. Однако если воспользоваться циклом чтение / обработка / запись на втором процессоре для записи значения в другую ячейку памяти, то на следующем шаге уже два процессора смогут прочитать это значение. Записав его затем еще в две ячейки, мы сможем воспользоваться уже четырьмя процессорами. В результате получается следующий алгоритм.

```
P[1] записывает значение в ячейку M[1]
procLoc = 1
for j = 1 to log p do
        Parallel Start
                for k = procLoc + 1 to 2 * procLoc do
                        P[k] читает M[k – procLoc]
                        P[k] записывает в M[k]
                end for k
        Parallel End
        procLoc = procLoc * 2
end for j
```

Сначала алгоритм записывает значение в ячейку M_1. При первом проходе внешнего цикла процессор P_2 читает это значение и записывает его в M_2, а переменная *procLoc* принимает значение 2. На втором проходе процессоры P_3 и P_4 читают содержимое ячеек M_1 и M_2 и записывают его в ячейки M_3 и M_4, а переменная *procLoc* принимает значение 4. На третьем проходе процессоры с P_5 до P_8 читают содержимое ячеек с M_1 до M_4 и записывают его в ячейки с M_5 до M_8. Ясно, что (в предположении, что p является степенью двойки) на предпоследнем шаге половина процессоров прочитает нужное значение и запишет его в ячейки с M_1 до $M_{p/2}$, после чего оставшаяся половина процессоров сможет его прочитать. Поскольку шаги чтения и записи можно выполнять в одном цикле, параллельный блок выполняет один цикл, а внешний цикл производит $\log p$ итераций. Поэтому этот параллельный алгоритм распределения выполняет $O(\log p)$ операций.

Поиск максимального элемента списка

Здесь и при других действиях над списками будем предполагать, что список содержится в ячейках памяти с M_1 до M_N. Кроме того предположим, что в нашем распоряжении $p = N / 2$ процессоров. (Случай $p < N / 2$ мы обсудим после разработки алгоритма.)

На первом проходе процессор P_i сравнивает значения в ячейках M_{2i} и M_{2i+1} и записывает в ячейку M_i большее из них. На втором проходе нам нужна только

половина процессоров, и с их помощью мы сравниваем пары элементов в ячейках с M_1 до $M_{N/2}$, записывая больший элемент пары в ячейки с M_1 по $M_{N/4}$. В результате получаем алгоритм, представленный ниже.

На каждом проходе цикла этого алгоритма число значений, среди которых может находиться максимальное, уменьшается вдвое пока не будет получено единственное число. Это очень похоже на реализацию метода турниров на единственном процессоре. Сам алгоритм выгладит следующим образом:

```
count = N / 2
for i = 1 to log(count) + 1 do
    Parallel Start
        for j = 1 to count do
            P[j] читает M[2j] в X и M[2j + 1] в Y
            if X > Y  P[j] пишет X в M[j]
            else  P[j] пишет Y в M[j]
            end if
        end for j
    Parallel End
    count = count / 2
end for i
```

Если $p < N / 2$, то можно предварительно уменьшить число значений до $2 * p$, а затем алгоритм выполняется так же, как и выше.

Видно, что число проходов цикла равно $\log N$, а его сложность имеет порядок $O(\log N)$. Напомним, о чем шла речь при обсуждении стоимости алгоритма в разделе 2.1.3. Мы определили стоимость как время работы, умноженное на число используемых процессоров. Поэтому стоимость нашего алгоритма равна $N / 2 * O(\log N)$, или $O(N \log N)$. Простой последовательный алгоритм выполняет ту же работу за $O(N)$ операций, поэтому он дешевле, однако параллельный алгоритм работает гораздо быстрее.

Если параллельные вычисления действительно могут принести пользу, то должен существовать альтернативный подход, не уступающий по стоимости последовательному. Посмотрев на предыдущий вариант повнимательней, можно увидеть, что число процессоров дает слишком большой вклад в стоимость. Можно ли как-то уменьшить это число? Если необходимо добиться того, чтобы полная стоимость была порядка $O(N)$, а время исполнения параллельного алгоритма – порядка $O(\log N)$, то число процессоров должно быть порядка $O(N / \log N)$. Это означает также, что на первом проходе каждый процессор должен обрабатывать $N / (N / \log N) = \log N$ значений. В результате приходим к следующему алгоритму.

```
Parallel Start
    for j = 1 to N / log N do
        P[j] находит максимум значений ячеек с M[1 + (j – 1) * log N]
              до M[j * log N] последовательным алгоритмом
        P[j] записывает найденный максимум в M[j]
    end for
Parallel End

count = (N / log N) / 2
for i = 1 to log (count) + 1 do
    Parallel Start
        for j = 1 to count do
            P[j] читает M[2j] в X и M[2j + 1] в Y
            if X > Y  P[j] пишет X в M[j]
            else P[j] пишет Y в M[j]
            end if
        end for j
    Parallel End
    count = count / 2
end for i
```

В этом варианте имеется этап предварительной обработки, на котором каждый процессор выполняет последовательный алгоритм на списке из log N элементов. Это требует $O(\log N)$ операций. Во второй части алгоритма воспроизведена наша исходная попытка, где число требуемых процессоров уменьшилось до (N / log N) / 2 (хотя на этапе предварительной обработки по-прежнему требуется N / log N процессоров). Полная стоимость алгоритма приведена в табл. 1.

Таблица 1. Полная стоимость алгоритма

Шаг	Стоимость	Время
Предварительная обработка	$(N / \log N) * O(\log N) = O(N)$	$O(\log N)$
Основной цикл	$[(N / \log N) / 2] * O(\log N) = O(N)$	$O(\log N)$
Всего	$O(N)$	$O(\log N)$

Стоимость последнего параллельного варианта алгоритма такая же, как и у последовательного алгоритма, однако работает он быстрее.